A “Humildade do Programador”

Das preocupações de Dijkstra à Programação Orientada a Objetos

Introdução ao Java

Table of contents

1 As Origens: 1972 **1**

2 Ferramentas Mentais contra a Complexidade **2**

3 O Problema do Código “Esparguete” **2**

4 Comparação: GOTO vs. Estrutura **2**
4.1 Abordagem GOTO (C) 2

5 O Labirinto do GOTO: Rastreio **3**

6 Comparação: GOTO vs. Estrutura **4**

7 Comparação: GOTO vs. Estrutura **4**

8 Comparação: GOTO vs. Estrutura **5**
8.1 Abordagem Java 5

9 How to set up **5**

10 Suggestions for IDE setup **6**

1 As Origens: 1972

Em 1972, **Edsger Dijkstra** proferiu o seu famoso discurso: *“The Humble Programmer”*.

> “Um bom programador reconhece que não é inteligente o suficiente para lidar com a complexidade bruta.”

A Mente Humana é Limitada: Dijkstra defendia que, como a nossa capacidade de processamento é finita, devemos escolher métodos que mantenham o software simples.

---

2 Ferramentas Mentais contra a Complexidade

Para gerir essa complexidade, Dijkstra propôs:

- **Separação de Preocupações:** Dividir para conquistar.
- **Abstração:** Criar camadas. Não precisamos de saber o que acontece "embaixo" para operar "em cima".
- **O Fim do GOTO:** O fluxo do programa deve ser acompanhável pela mente humana.

---

3 O Problema do Código "Esparguete"

O comando `GOTO` criava saltos lógicos que tornavam o código impossível de seguir.

Vamos comparar o desafio de lógica proposto por Dijkstra: o Caos (C/GOTO) vs. a Ordem (Java Estruturado).

---

4 Comparação: GOTO vs. Estrutura

4.1 Abordagem GOTO (C)

```
10: A = 10;
20: B = 2;
30: GOTO 70;
40: IF A < 25 GOTO 90;
50: B = B + A;
60: GOTO 110;
70: B = B * 2;
80: GOTO 40;
```

```
90: A = A + 5;
100: GOTO 70;
110: PRINT B;
```

---

5 O Labirinto do GOTO: Rastreio

Acompanhar este fluxo manualmente é o que Dijkstra chamava de "carga cognitiva desnecessária".

Passo	Linha	A	B	O que aconteceu?
1	10, 20	10	2	Variáveis inicializadas.
2	30	10	2	**GOTO 70** (Salta para processamento de B).
3	70	10	4	B é duplicado (2 * 2).
4	80	10	4	**GOTO 40** (Salta para verificação).
5	40	10	4	$10 < 25$? Sim. **GOTO 90**.
6	90	15	4	A aumenta para 15.
7	100	15	4	**GOTO 70** (Ciclo recomeça).
8-13	...	20	16	(Repetição do ciclo: B duplica, A aumenta).
14-19	...	25	32	(Última repetição: B chega a 32, A chega a 25).

Passo	Linha	A	B	O que aconteceu?
20	40	25	32	$25 < 25$? **Não**. Segue para linha 50.
21	50	25	57	B = 32 + 25 (Soma final).
22	60	25	57	**GOTO 110** (Fim do programa).
23	110	25	**57**	**Resultado Final: 57**

---

6 Comparação: GOTO vs. Estrutura

Resultado Final: O programa com GOTO é difícil de seguir, exigindo um rastreio manual complexo para entender o fluxo e o estado das variáveis.

Passo	Linha	A	B	O que aconteceu?
23	110	25	**57**	**Resultado Final: 57**

---

7 Comparação: GOTO vs. Estrutura

Conclusão: No Java, este "vaivém" é substituído por um `while` legível, onde o início e o fim do bloco são claros.

---

8 Comparação: GOTO vs. Estrutura

8.1 Abordagem Java

```java
public class DesafioDijkstra {
    public static void main(String[] args) {
        int a = 10;
        int b = 2;

        // O primeiro passo do nosso antigo 'GOTO'
        b = b * 2;

        // O loop 'while' substitui os saltos manuais
        while (a < 25) {
            a += 5;
            b *= 2;
        }

        b += a;

        System.out.println("Resultado Final: " + b);
    }
}
```

---

9 How to set up

Install IntelliJ

https://lp.jetbrains.com/intellij-idea-promo/

Use UC account to create a student license

---

10 Suggestions for IDE setup

How to set up:

- File > New > Project.
- Select Java (or your language) and name it something like Programming_Basics.
- Build System: Choose IntelliJ. It's simpler for beginners than Maven or Gradle.
- JDK: If you don't have one, click "Download JDK." Version 21 or 17 (LTS) are great choices.
- Once the project opens, right-click the src folder in the left sidebar.
- Select New > Package and name it day1_basics.
- Right-click that package and select New > Java Class.